

制御教育教材 StampFly Ecosystem の紹介

コーディングから飛行試験、データ取得まで

伊藤 恒平（金沢工業大学）

2026 年 9 月 10 日（木）

システム制御情報学会・計測自動制御学会 チュートリアル講座
2026

伊藤 恒平 — 金沢工業大学

StampFly Ecosystem の開発者。専門はドローンの飛行制御工学

- 本チュートリアルで扱うファームウェア・シミュレータ・教材は、すべて本人が開発と保守を行っている
- 大学の講義、高校教員向け講座、本日のような学会チュートリアルなど、複数の教育現場で同じコードベースを使い回している

本日の進め方 / How Today Works

講義とデモのハイブリッド

講師が実演しながら説明を進める。手元の環境構築でつまずいても、その場で全員を待たせることはしない

- デモは「見るだけ」でも十分に内容を追える構成にしてある
- 手を動かしたい方は、講師と同じコマンドと一緒に打ってみてほしい
- 進度が合わなかった箇所は、録画と資料を使って帰宅後に復習できる

各セッション共通の型

地図（今どこにいるか） → 要点3つ → デモ → 理論とコードの対応表 → 復習パス → チェックポイント

デモの3段階

- **見るだけ:** 画面とスクリーンを見ているだけで流れがわかる
- **一緒に打つ:** 持参の PC と StampFly でコマンドを再現する
- **帰宅後に再現:** 会場では見るだけにして、録画と資料で後日試す

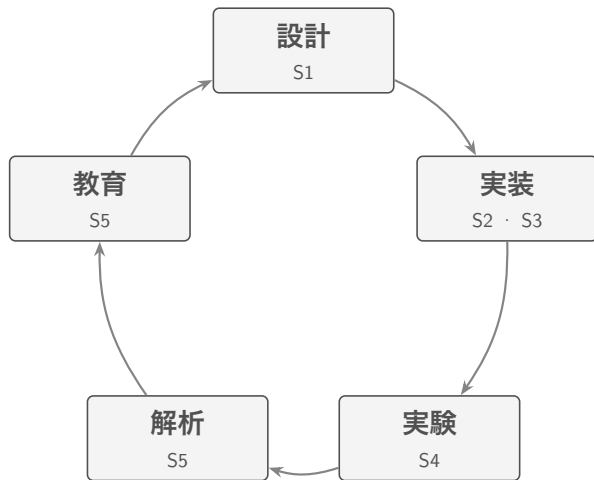
資料はすべて公開されている

リポジトリ: https://github.com/M5Fly-kanazawa/stampfly_ecosystem

ドキュメント: https://m5fly-kanazawa.github.io/stampfly_ecosystem/docs/

本日の内容はオンデマンド配信でも後日視聴できる

今日の地図 / Today's Map



StampFly Ecosystem は「設計 → 実装 → 実験 → 解析 → 教育」を循環させながら育ってきた。本日の5セッションはこの循環の上を進む

S1	10:05 – 11:00
S2	11:00 – 12:00 (昼休み)
S3	13:00 – 14:00
S4	14:00 – 15:00 (休憩)
S5	15:30 – 16:30

必要機材と安全ルール / Equipment & Safety

持参いただくもの

StampFly 実機とコントローラ、ノート PC（実習に参加する場合）

飛行時の安全ルール

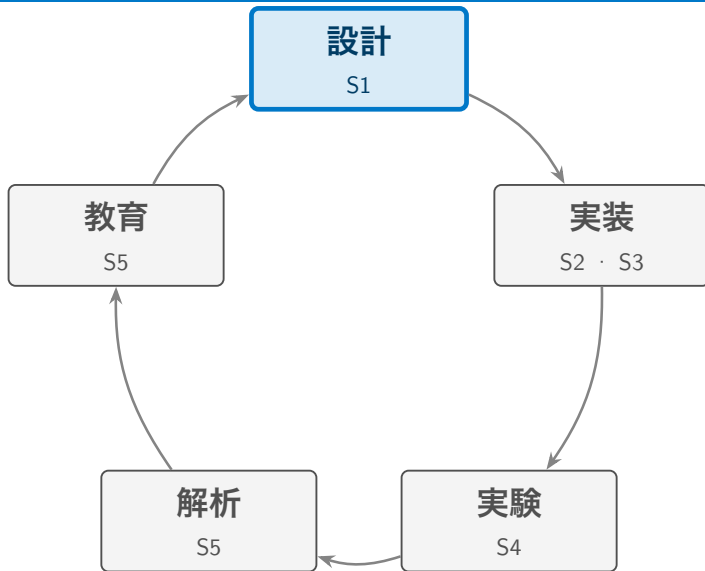
- プロペラガードを装着した状態で飛行させる
- 飛行エリアはネットで区画された範囲に限る
- **ARM** は機体ボタンの単クリックまたはコントローラで行う。予期せず回転したら即座に **DISARM**
- 機体の真上には顔や手を出さない

Session 1

StampFly Ecosystem の全体像と設計思想

10:05 – 11:00

SCI/SICE チュートリアル講座 2026



要点3つ / Three Takeaways

- ① マルチコプタは制御なしには1秒も飛べない。この性質が制御工学の教材として理想的である
- ② 必要なリソースが分散していることが普及の障壁になっている。StampFly Ecosystem は単一リポジトリへの集約でこれに応える
- ③ ファームウェアは4階層のアクセス構造を持ち、Workshop 受講者から研究者まで同じコードベースを共有しながら自分の関心に合わせて深さを選べる

制御がなければ飛べない / No Control, No Flight

マルチコプタは固定翼機のように滑空できず、ヘリコプタのようにオートローテーションで降りることもできない。制御系が止まれば即座に墜落する

教材としての魅力

この性質は安全上の課題であると同時に、学習者が「制御がなければ飛べない」という事実を身をもって体験できる特性でもある。フィードバック制御の役割を直感的に理解する入口になる

StampFly ハードウェア / Hardware

項目	仕様
MCU	ESP32-S3 (M5StampS3)
IMU	BMI270 (6 軸 400 Hz)
気圧	BMP280
磁力計	BMM150 (3 軸)
測距	VL53L3CX ×2 (Bottom / Front)
光学フロー	PMW3901
電流電圧	INA3221
モータ	コアレスモータ ×4
バッテリー	300 mAh 1S LiPo
質量	37 g (バッテリー含む)
コントローラ	M5Atom JoyStick (ESP-NOW)

4つの障壁 / Four Barriers

手のひらサイズで安全性が高く安価な StampFly でも、教育現場での普及にはリソース分散に加えて次の障壁がある

- **ブラックボックス問題:** 市販の教育用ドローンは飛ぶが、制御ループの中身に触れられない
- **理論と実装のギャップ:** PID の数式は教科書で学べても、組み込みのリアルタイム制御ループとして実装するには別のスキルが要る
- **体系的教材の不在:** センサ取得から状態推定、制御、実験、データ解析までを一貫して学べる教材が見当たらない
- **教育者の準備負担:** カリキュラム設計・教材作成・ツール整備を一人の教育者がすべて担うのは現実的でない

既存プラットフォームとの比較 / Platform Comparison

	Crazyflie	Tello EDU	PX4	StampFly Eco.
制御ループアクセス	✓	—	✓	✓
教材・カリキュラム	△	△	△	✓
シミュレータ統合	✓	—	✓	✓
コスト	高	低	高	低
コード規模	中	—	大	小

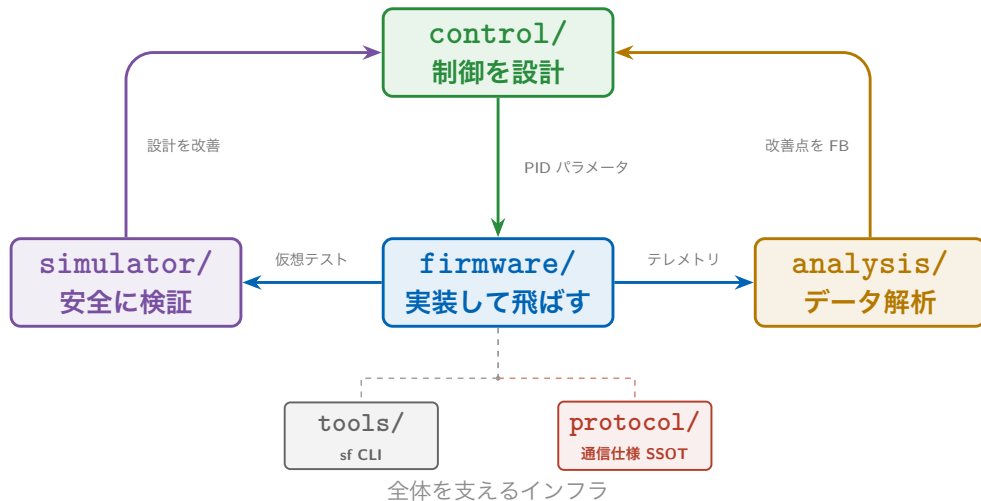
制御ループへのフルアクセスと体系的な教材・ツールの統合提供を両立させ、ファームウェア全体を一人の学習者が読み通せる規模に抑えている点が異なる

1 機から学べる 8 分野 / Eight Fields, One Airframe

分野	主な学習項目
制御理論	PID, カスケード制御, FF/FB, アンチwindアップ
飛行力学	6DoF 運動方程式, ホバー条件, NED/Body 座標変換
センサ工学	IMU, 気圧, 光学フロー, Allan 分散, ノイズ解析
状態推定	相補フィルタ, ESKF, センサフュージョン
アクチュエータ	PWM 制御, 推力モデル, プロペラ空力特性, ミキサ行列
システム同定	周波数応答, パラメータ推定, モデル検証
通信・IoT	ESP-NOW, WiFi テレメトリ, プロトコル設計
ソフトウェア工学	リアルタイムタスク, 組込み開発, OSS 開発

これらは独立した知識ではなく、1 台の機体の中で密接に関連し合っている。PID ゲインの設計には慣性モーメントやモータ応答特性の知識が要り、推定精度はセンサのノイズ特性に左右される

エコシステムの地図 / Ecosystem Map



設計思想 5 本柱 / Five Design Pillars

柱	内容
集約と Protocol as SSOT	ファーム・シミュレータ・解析・教材を単一リポジトリに集約。ESP-NOW 通信仕様は <code>protocol/spec/</code> を唯一の基準とする
sf CLI	ビルド・書込み・ログ取得・シミュレーション操作を統一コマンド体系で提供
段階的抽象化 WS::	Workshop API が HW・タスク・Topic の詳細を隠しつつ、レッスンの進行に応じて段階的に開放する
シミュレータ統合	VPython (軽量 3D) と Genesis (高精度物理) を、実ファームと同一の制御アルゴリズムで動かす
オープンソース	制御ループからドライバまで公開し、ブラックボックス問題を根本から排除する

深掘り: sf CLI / Deep Dive: sf CLI

idf.py を直接叩く代わりに、ビルド・書き込み・診断・ログ取得・シミュレーションを統一されたコマンド体系で提供する

カテゴリ	コマンド例
ビルド・書き込み	<code>sf build / sf flash</code>
診断	<code>sf doctor / sf monitor</code>
ログ	<code>sf log wifi / sf log viz</code>
シミュレータ	<code>sf sim run / sf sils gui</code>
同定・整合	<code>sf sysid / sf params check</code>

深掘り：段階的抽象化 / Deep Dive: Progressive Abstraction

ws:: 名前空間は、制御ループの起動やセンサ初期化を隠しつつ、`setup()` と `loop_400Hz(dt)` だけで機体を動かせるようにする

- 隠蔽は永続的なブラックボックスではなく、レッスンの進行に応じて解除される
- 初期のレッスンはモータ制御 API のみ公開し、進むにつれてセンサ・推定値・テレメトリが順次開放される
- この考え方をファーム全体に一般化したものが、次に示す 4 階層アクセスである

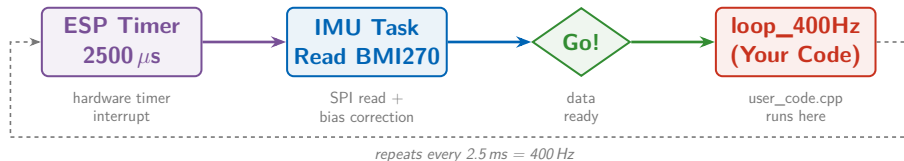
vehicle ファーム構造 / vehicle Firmware Structure

コンポーネント数	30 (sf_* 命名, フラット構造)
通信方式	Pub-Sub トピック (直接呼び出し禁止)
タスク数	14
制御周期	400 Hz (IMU → 状態推定 → 制御)
状態推定	ESKF (誤差状態カルマンフィルタ)
制御器	カスケード PID
飛行モード	ACRO / STABILIZE / ALT_HOLD / POS_HOLD

実機検証

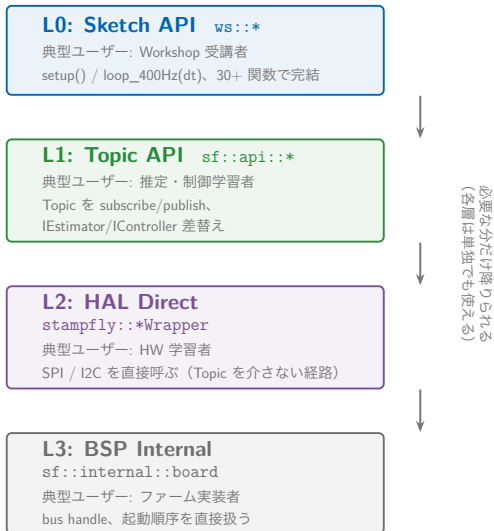
POS_HOLD (位置制御) は実機で検証済み。保持精度 $\pm 6\text{--}7\text{ cm}$

400 Hz 制御ループ / The 400 Hz Control Loop



IMU 割込みを起点に、状態推定 → 制御 → アクチュエーションが 2.5 ms 周期で走る。全コンポーネントが同じ周期を共有する

4 階層アクセス / Four-Tier Access



シミュレーション3層 / Three Simulation Layers

層	内容	位置づけ
層 1 設計用線形モデル	実飛行ログから同定した低次 $G(s)$	制御系設計・ループ整形の基準
層 2 実ログ駆動再生	移植した制御則を実ログの外乱・指令で駆動	パラメータ変更の A/B 判定の基準
層 3 SILS	MuJoCo + 実ファーム C++、Code/-Param Identity	システム全体の検証ベンチ

層 3 の合否はモデル一致ゲートで判定する。詳細は S5 で扱う

開発の3原則 / Code Identity, Parameter Identity, Model Fidelity

原則	内容
Code Identity	SILS は本体ソースを書き換えず参照コンパイルして実行する。一致するのは推定・制御の数式だけでなく、ループ全体である
Parameter Identity	SILS も実機も <code>params.cpp</code> を唯一の基準として読む。チューニング値は同じスキーマで往復する
Model Fidelity	SILS の真値は物理モデルから得る。実機データが必要になるのは、初飛行のあとにモデルを現実へ近づける段階だけである

原稿提出後 5 か月の進捗 / Five Months Since the Manuscript

本チュートリアル の原稿 (SCI'26 OS04) は 2026 年 4 月に提出した。それ以降に進んだことを紹介する

- MuJoCo ベース SILS。42 本のシナリオのうち 32 本を CI で回帰 (`sf sils regression`)
- 対話的シナリオ実行・可視化ツール `sf sils gui`
- POS_HOLD の実機検証 (保持精度 $\pm 6\text{--}7\text{ cm}$)
- ビルド不要の Web Flasher と GUI インストーラ「StampFly Setup」
- Tello 互換 Python SDK (`lib/stampfly`) と `sf flight` (`takeoff/land/hover/jump`)
- Workshop ファームの vehicle コンポーネント基盤への統合

デモ①: シミュレータ操縦 / Demo 1: Simulator Piloting

見るだけ | 一緒に打つ | 帰宅後に再現

何を見るか

- `sf sim run` で VPython シミュレータが起動し、実機と同じ制御アルゴリズムが動く
- HID ジョイスティック (Atom JoyStick) でそのまま操縦できる
- 実機がなくても制御コードの挙動を 3D で確認できる

デモ②: 実機 POS_HOLD 飛行 / Demo 2: Real POS_HOLD Flight

見るだけ | 一緒に打つ | 帰宅後に再現

何を見るか

- ホバー中の位置保持精度 (実測 $\pm 6-7$ cm)
- 手で軽く押すなどの外乱を与えたときの復帰動作
- 会場の条件に備え、事前撮影の動画を保険として用意する

形態	内容
4+1 日ワークショップ	Day1-4 で Lesson 0-12、Day5 午前に精密着陸競技会
大学講義シラバス	半期 15 回 (各 90 分)、工学系学部 3-4 年生向け
高校教員向け講座	DXH 講座 (2026 年 7 月、操縦体験と書き込み実習)

復習パス / Review Path

コマンド	対象	文書	Notebook
<code>sf sim run</code>	シミュレータ操縦	<code>docs/architecture/simulation-policy.md</code>	<code>01_hello_stampfly.ipynb</code>
<code>sf sils gui</code>	SILS 回帰・可視化	<code>firmware/vehicle/docs/development_roadmap.md</code>	—
—	4階層アクセス	<code>firmware/vehicle/docs/architecture.md</code> §2	—
—	大学講義シラバス	<code>docs/university/syllabus.md</code>	全15冊

チェックポイント / Checkpoint

確認事項

- ☐ マルチコプタが制御教材として適している理由を説明できる
- ☐ StampFly Ecosystem が集約によってどの障壁に対応しているか説明できる
- ☐ 4階層アクセス（L0-L3）のうち自分の関心に合う層を1つ選べる

次のセッション

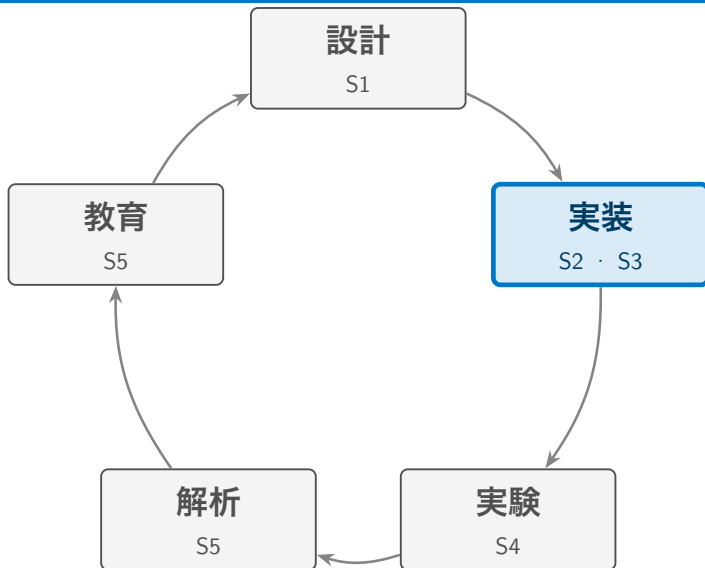
S2: 開発環境のセットアップとセンサデータの取得（11:00 – 12:00）

Session 2

開発環境のセットアップ とセンサデータの取得

11:00 – 12:00

SCI/SICE チュートリアル講座 2026



要点3つ / Three Takeaways

- ① ビルド環境は ESP-IDF と `sf CLI` が担う。開発を伴わない参加者は Web Flasher / GUI インストーラという代替経路がある
- ② 学習者が書くコードは `setup()` と `loop_400Hz(dt)` の2関数だけ。センサやHWの詳細は `ws::` 名前空間が隠す
- ③ センサデータの取得経路は3通りある。目的に応じて `ws::print`、`sf telemetry`、`sf log wifi` を使い分ける

事前準備の確認 / Environment Check

全員で `sf doctor` を実行する

ESP-IDF、Python、USB シリアルドライバ、sf CLI 自体のバージョンをまとめて診断する

```
source setup_env.sh  
sf doctor
```

ここで通らなくても心配は不要

すべて OK にならなかった場合は、このあとのデモは「見るだけ」で参加し、復習パス（本セッション末尾）に沿って会場外で環境を整えてほしい

ESP-IDF の上に sf CLI を重ねる

idf.py を直接叩く代わりに、ビルド・書き込み・診断・ログ取得を統一したコマンド体系で提供する

コマンド	役割
<code>sf build [target]</code>	ファームウェアビルド
<code>sf flash [target] -m</code>	書き込み (-m でモニタ付き)
<code>sf monitor</code>	シリアルモニタ
<code>sf doctor</code>	環境診断

target には vehicle / controller / workshop を指定する

ビルド不要の選択肢 / No Dev Environment Needed

開発環境がなくても書き込みだけ是可以る

ブラウザまたはデスクトップアプリから、公開済みの完成版ファームウェアを直接書き込める

方法	特徴
Web Flasher	ブラウザ（Chrome/Edge）から書き込み。インストール不要
GUI インストーラ	デスクトップアプリ「StampFly Setup」。ウィザードに従うだけで sf CLI 一式を導入

本チュートリアルの実習は sf CLI を使うが、授業に持ち帰る際は受講者の環境に応じて使い分けてほしい

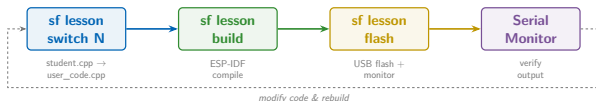
Sketch の型とワークフロー / Sketch Shape & Workflow

学習者が書く関数は2つだけ

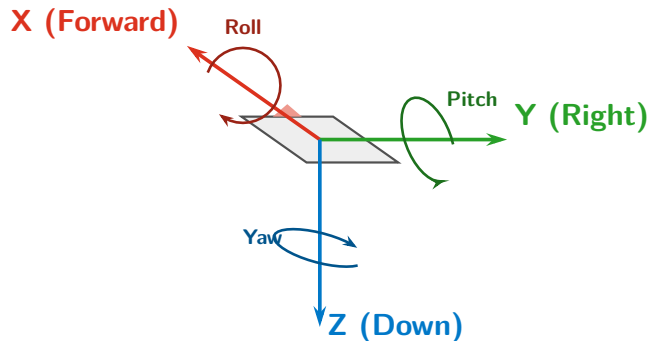
- `setup()` — 起動時に 1 回
- `loop_400Hz(dt)` — 400 Hz 毎

切替・ビルド・書き込み

```
sf lesson switch N  
sf lesson build  
sf lesson flash
```



IMU の軸定義 / IMU Axis Convention



BMI270 — 6 軸 (3 軸ジャイロ + 3 軸加速度) 400 Hz | Body FRD 座標系

IMU API と単位 / IMU API & Units

関数	説明	単位
<code>gyro_x/y/z()</code>	角速度 (Roll/Pitch/Yaw)	rad/s
<code>accel_x/y/z()</code>	加速度 (X/Y/Z)	m/s ²

静止時の目安

ジャイロ ≈ 0 、加速度 $Z \approx -9.81 \text{ m/s}^2$ (重力に対する反力)

データ取得経路① Teleplot / Data Path 1: Teleplot

ws::print をそのままグラフにする

>変数名: 値 の形式で出力すると、VSCode 拡張機能 Teleplot がリアルタイムにグラフ化する

user_code.cpp

```
1 void loop_400Hz(float dt) {  
2     if (tick++ % 4 == 0) {    // 100Hz decimation  
3         ws::print(">gyro_x:%.3f", ws::gyro_x());  
4         ws::print(">gyro_y:%.3f", ws::gyro_y());  
5     }  
6 }
```

sf monitor で接続し、VSCode の Teleplot パネルを開くだけで波形が動き出す

データ取得経路② sf telemetry / Data Path 2: sf telemetry

コード変更なしでライブ表示

機体は状態推定値を常時 50 Hz で送信している。受信して表示するだけなら学習者コードの変更は不要

```
sf telemetry          # ターミナル表示（既定）
sf telemetry --web     # ブラウザ表示（UDP:5005 -> SSE）
```

講義や複数人での画面共有には --web が見やすい

データ取得経路③ ログ取得と可視化 / Data Path 3: Log & Visualize

WiFi 経由で 400 Hz Data Stream を記録する

`sf log wifi` は機体と同じ WiFi (SoftAP モードなら機体が出す AP) で 400 Hz の Data Stream を UDP 受信し、`-o *.csv` で 1 周期 1 行の CSV に保存する

```
sf log wifi -d 30 -o flight.csv    # 30秒 キャプチャ -> Data Stream CSV
sf log viz flight.csv
```

使い分けの目安

Teleplot = その場で波形を見る / `sf telemetry` = コード変更なしで確認 / `sf log wifi` = 後で解析するためのデータ保存

デモ: 傾けて確認 / Demo: Tilt Test

見るだけ | **一緒に打つ** | 帰宅後に再現

何を見るか

- StampFly を手に持ち、前後左右に傾ける
- `gyro_x/gyro_y` が傾ける速さに応じて振れる
- 静止させると `accel_z` が -9.81 付近に戻る

手順

```
sf lesson switch 4 → 前ページのコードを書く → sf lesson build && sf lesson flash → Teleplot で確認
```

他のセンサ / Other Sensors

IMU 以外のセンサも同じ考え方で読める。時間の都合で本セッションでは扱わないが、単独ビルド可能な例題として用意してある

例題	センサ
examples/05_read_tof	VL53L3CX (ToF 距離センサ)
examples/06_read_baro	BMP280 (気圧センサ)

firmware/vehicle/examples/ 配下。各ディレクトリで `idf.py build` すれば単体で動く

理論とコードの対応表 / Theory-to-Code Map

理論	コード	Data Stream 列	トピック
角速度 ω [rad/s] Body FRD	<code>ws::gyro_x()</code>	gyro_x	sensor_imu
並進加速度 a [m/s ²]	<code>ws::accel_x()</code>	accel_x	sensor_imu

学習者が呼ぶ `ws::` 関数は、内部で `sensor_imu` トピックを購読しているだけである (L0 $\rightarrow$ L1 の対応)

復習パス / Review Path

コマンド	レッスン	文書	Notebook
<code>sf lesson switch 0</code>	Lesson 0	docs/guides/troubleshooting.md	—
<code>sf lesson switch 4</code>	Lesson 4	firmware/vehicle/docs/architecture.md	<code>01_hello_stampfly.ipynb</code>
<code>sf log wifi / sf log viz</code>	—	docs/guides/flight-log-viz.md	<code>15_analysis_toolkit.ipynb</code>

チェックポイント / Checkpoint

確認事項

- ☐ `sf doctor` が通る、または代替経路 (Web Flasher / GUI インストーラ) を把握した
- ☐ `setup()/loop_400Hz(dt)` の2関数構造を説明できる
- ☐ ジャイロ・加速度をどれか1つの経路で確認できた

次のセッション

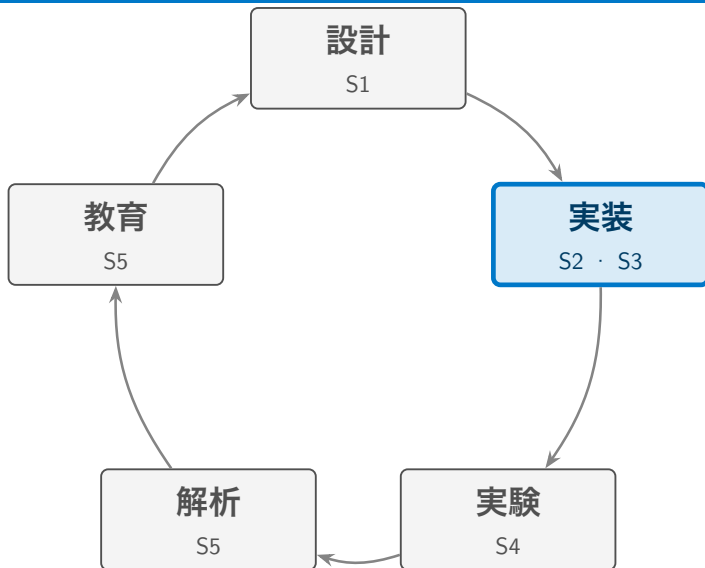
S3: モータ制御とコントローラ入力の実装 — センサの次は、機体を動かす側を作る

Session 3

モータ制御とコントローラ実装

13:00 – 14:00

SCI/SICE チュートリアル講座 2026



要点3つ / Three Takeaways

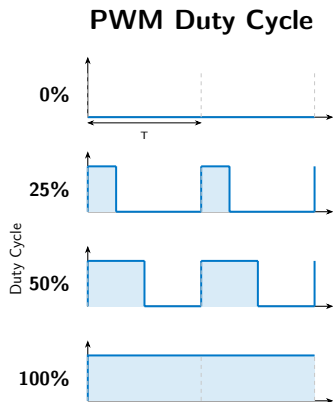
- ① モータは PWM の Duty 比で駆動する。4 基の配置と回転方向はトルクバランスで決まっている
- ② コントローラのスティック値は ESP-NOW で 14 バイトの `ControlPacket` として届く
- ③ `ws::motor_mixer(T,R,P,Y)` で手動ミキシングしても、フィードバックがない限り安定して飛べない。ここに次セッションの動機がある

モータを回す実習の注意

- **プロペラは外した状態**で行う（本セッションは机上でモータの回転だけを確認する）
- **Duty は 0.15 以下**に固定する。ファーム側では強制されないので数値を必ず確認する
- **ARM** は機体ボタンの単クリックまたはコントローラで行う。コードの中で自動 ARM しない

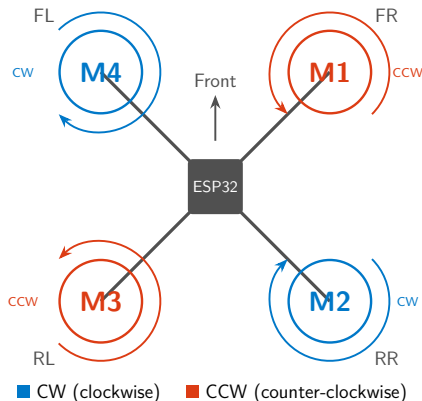
止め方: 機体ボタンをもう一度クリック (DISARM)。バッテリー駆動での確認を基本とする

モータと PWM / Motors & PWM



- LEDC (ESP32 のハードウェア PWM) でモータを駆動する
- Duty 0% = 停止、100% = 最大回転
- API: `ws::motor_set_duty(id, duty)`
id=1-4, duty=0.0-1.0

モータ配置 / Motor Layout



- モータ ID: 1=FR (右前)、2=RR (右後)、3=RL (左後)、4=FL (左前)
- 対角のモータが同じ方向に回転する。M1・M3 = CCW、M2・M4 = CW (反トルクを打ち消す)

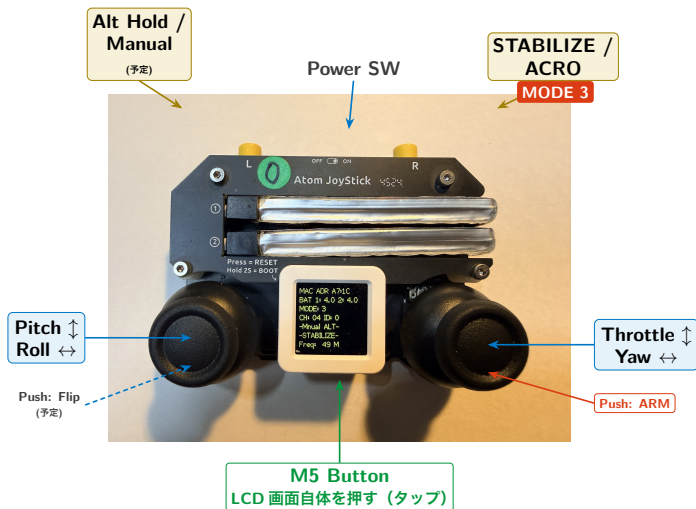
実習: Duty をハードコードして回す / Hands-on: Hardcoded Duty

user_code.cpp

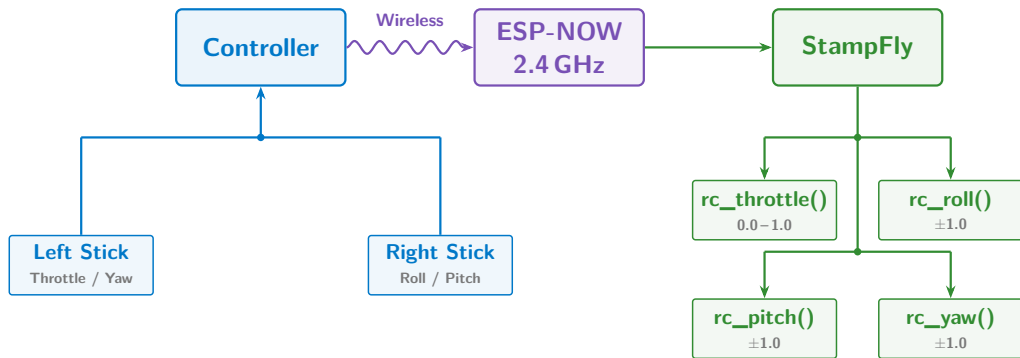
```
1 void setup() {
2     ws::print("Motor duty test");
3     // Do NOT auto-arm in code / コードでは自動ARMしない
4 }
5 void loop_400Hz(float dt) {
6     ws::motor_set_duty(1, 0.10f);    // FR
7     ws::motor_set_duty(2, 0.00f);    // RR
8     ws::motor_set_duty(3, 0.00f);    // RL
9     ws::motor_set_duty(4, 0.00f);    // FL
10 }
```

sf lesson switch 1 で切替。ARM は機体ボタンの単クリックで行う

コントローラ / Controller



ESP-NOW と ControlPacket / ESP-NOW & ControlPacket



コントローラと StampFly は ESP-NOW で直接通信する。1 パケット
ControlPacket は **14 バイト** (throttle/roll/pitch/yaw + flags + checksum)

ペアリングとチャンネル / Pairing & Channel

ペアリング

コントローラの M5 ボタン (LCD パネル) を押しながら電源 ON → StampFly のボタンを約 2 秒長押し → 両方のビープで完了。次回以降は自動接続

チャンネル設定

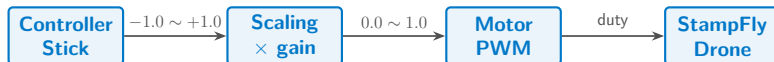
教室内で複数機体が同時に飛ぶと混信する。`ws::set_channel(ch)` を `setup()` で呼んで機体のチャンネルを 1/6/11 のいずれかにし、再ペアリングする。コントローラはペアリング時に機体のチャンネルへ自動追従する

自分で購入したコントローラを使う場合

ご購入直後のコントローラには出荷時点のファームウェアが入っているが、本チュートリアルで使う版とは異なる。最初に一度だけ、`sf flash controller` で書き込み直す必要がある

オープンループ制御 / Open-Loop Control

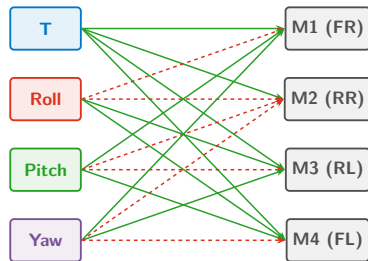
Open-Loop Control



No Feedback!

<code>ws::rc_throttle/roll/pitch/yaw()</code>	スティック値の取得
<code>ws::motor_mixer(T, R, P, Y)</code>	4 モータへの配分

ミキサ行列 / Mixer Matrix



Pitch+ → 前傾 → 前 (M1,M4) 強 + 後 (M2,M3) 弱

Roll+ → 右傾 → 右 (M1,M2) 弱 + 左 (M3,M4) 強

意味

4 行 4 列の行列は「T/R/P/Y の指令を 4 モータの Duty にどう配分するか」を表す。行ごとの符号がモータ配置 (motor_layout) と対応する

$$\begin{bmatrix} M_1 \\ M_2 \\ M_3 \\ M_4 \end{bmatrix} = \frac{1}{4} \begin{bmatrix} 1 & -\frac{1}{L} & +\frac{1}{L} & +\frac{1}{k_q} \\ 1 & -\frac{1}{L} & -\frac{1}{L} & -\frac{1}{k_q} \\ 1 & +\frac{1}{L} & -\frac{1}{L} & +\frac{1}{k_q} \\ 1 & +\frac{1}{L} & +\frac{1}{L} & -\frac{1}{k_q} \end{bmatrix} \begin{bmatrix} T \\ \tau_\phi \\ \tau_\theta \\ \tau_\psi \end{bmatrix}$$

— + positive
--- - negative

デモ: スティックでモータを回す / Demo: Stick to Motors

見るだけ | **一緒に打つ** | 帰宅後に再現

手順

sf lesson switch 2 → ペアリング → ws::motor_mixer を書いて sf lesson build
&& sf lesson flash → スティックで4モータの回転差を確認

これでは飛べない理由

プロペラを外した状態でも、風で軽く煽るだけで指令通りの回転が保てないことが見て取れる。外乱に対して何の補正もかからないためである

プラントの定義 / Defining the Plant

本セッションで作った経路が、そのまま制御対象（プラント）の定義になる

入力	<code>ws::motor_mixer</code> が出す Duty 指令（トルク指令に相当）
出力	<code>ws::gyro_x/y/z()</code> が返す角速度の実測値

次セッションへ

この入出力の間にどんな伝達関数があり、どんなゲイン K_p を選べば安定して追従するか。それが S4「フィードバック制御の基礎」の主題である

理論とコードの対応表 / Theory-to-Code Map

理論	コード	通信	トピック
Duty 比 $[0, 1]$	<code>ws::motor_set_duty(id,d)</code>	—	<code>actuator.motor</code>
スティック指令 T, R, P, Y	<code>ws::rc_throttle()</code> 等	ControlPacket (14B)	<code>command_setpoint</code>
ミキシング	<code>ws::motor_mixer(T,R,P,Y)</code>	—	—

復習パス / Review Path

コマンド	レッスン	文書	Notebook
sf lesson switch 1	Lesson 1 モータ制御	firmware/vehicle/docs/hardware_init.md	—
sf lesson switch 2	Lesson 2 コントローラ入力	protocol/spec/ (ControlPacket)	—
sf flash controller	—	docs/commands/sf-flasher.md	—

チェックポイント / Checkpoint

確認事項

- ☐ Duty とモータ配置・回転方向の対応を説明できる
- ☐ ControlPacket がどの経路 (ESP-NOW) でどれだけの頻度で届くか説明できる
- ☐ オープンループ制御が「なぜ飛べないか」を自分の言葉で説明できる

次のセッション

S4: フィードバック制御の基礎 — PID による姿勢安定化 (14:00 – 15:00)

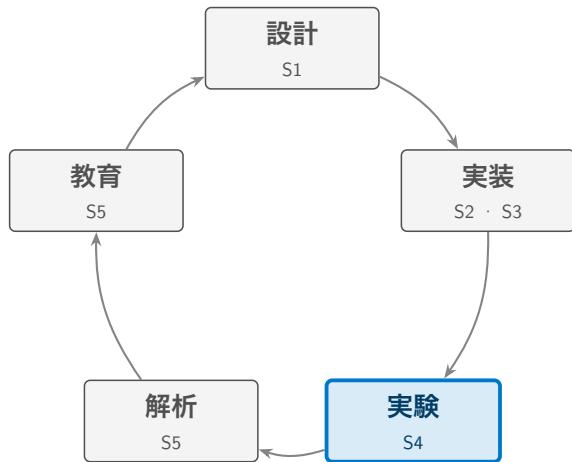
Session 4

フィードバック制御の基礎 — PID による姿勢安定化

14:00 – 15:00

SCI/SICE チュートリアル講座 2026

地図: 今ここ / Map: You Are Here



S2・S3 で組み上げた機体を、本セッションでは制御理論と突き合わせて飛ばす。「設計 → 実装」の次にくる「実験」のフェーズにあたる。

今日のセッションで扱うもの

レート P 制御の初飛行 → プラントモデルによる裏付け → PID 化 → 姿勢推定 → 実機データでの検証

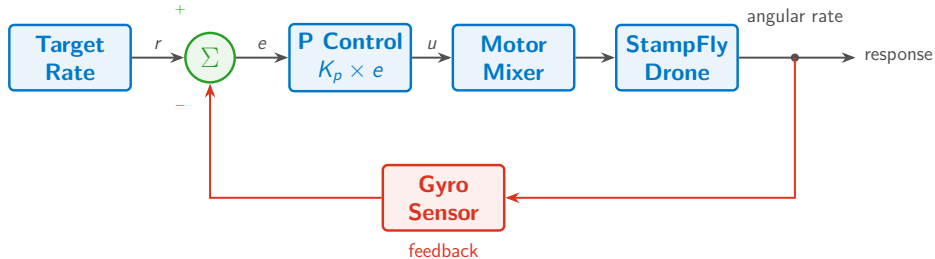
要点3つ / Three Key Points

- ① モータ+機体のプラント伝達関数 $G_p(s) = K/(s(\tau_m s + 1))$ を実測パラメータから導出し，ゲイン設計に使う
- ② P 制御から PID 化し，不完全微分・アンチwindアップ・積分リセットまで含めた vehicle 実装と学習者コードを対比する
- ③ フライトデータからプラントを同定し，設計値と実測値を突き合わせる

本セッションの立ち位置

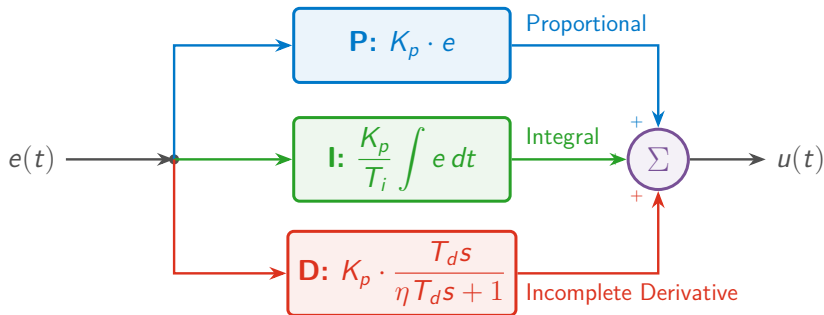
ここまでのレッスンは動かすことが目的だった。ここからは，動いた結果をモデルと数値で説明できるかを問う

内側ループ: レート制御 / Inner Loop: Rate Control



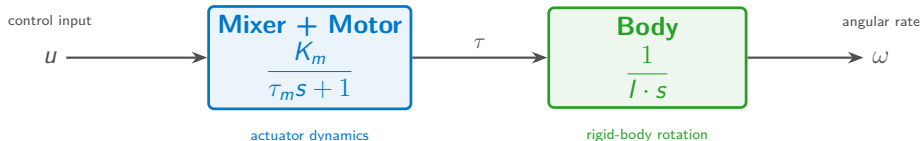
- この図は**内側ループ（レート制御）**：目標角速度とジャイロ計測値の差を制御。ACRO はこのループ単体で完結する
- **外側ループ（姿勢制御）** は、この図の入力 Target Rate を作る側にもう1段乗る。目標姿勢角と推定角の差から目標角速度を作り、STABILIZE 以上で有効になる（カスケード = ループの入れ子）

PID 制御器 / PID Controller



$$C(s) = K_p \left(1 + \frac{1}{T_i s} + \frac{T_d s}{\eta T_d s + 1} \right)$$

プラントモデル / Plant Model



$$G_p(s) = \frac{K}{s(\tau_m s + 1)}, \quad K = \frac{K_m}{I}$$

ホバー近傍で線形化すると2ブロックに集約できる

- **Mixer + Motor:** duty $\rightarrow$ トルク, 1次遅れ $K_m/(\tau_m s + 1)$
- **Body:** トルク $\rightarrow$ 角速度, 積分器 $1/(I \cdot s)$

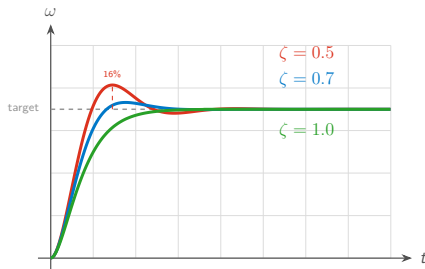
実測パラメータ / Measured Parameters

パラメータ	記号	Roll	Pitch	Yaw
慣性モーメント	I	9.16e-6	13.3e-6	20.4e-6 kg·m ²
トルクゲイン	K_m	9.3e-4	9.3e-4	1.6e-4 N·m
モータ時定数	τ_m	0.02 s (共通, 実測 0.0163 s)		
プラントゲイン	$K = K_m/I$	102	70	8.0 rad/s²

軸ごとに K が違う理由

Roll/Pitch は同じ K_m だが $I_{yy} > I_{xx}$ なので Pitch の方が遅い。Yaw は $\kappa \ll L$ (トルク推力比がアーム長よりずっと小さい) ため K_m 自体が小さく、さらに遅い

2次系設計 / Second-Order Design



閉ループ伝達関数 (2次系)

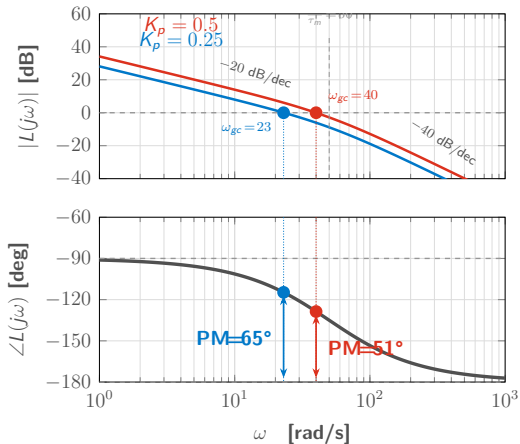
$$G_{cl}(s) = \frac{K_p K}{\tau_m s^2 + s + K_p K}$$

ζ から K_p を逆算する設計式

$$K_p = \frac{1}{4\zeta^2 K \tau_m}$$

Roll/Pitch で $\zeta = 0.7$ (オーバーシュート約5%)
を狙うと $K_{p,roll} = 0.25$, $K_{p,pitch} = 0.36$

開ループボード線図 / Open-Loop Bode Plot

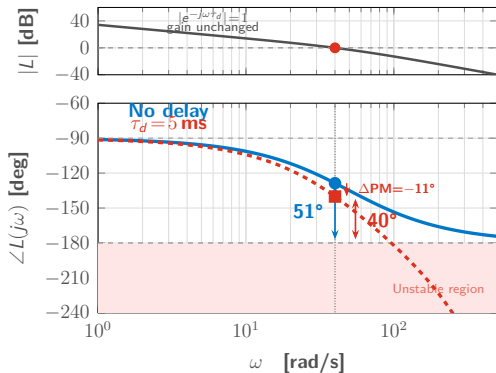


開ループ伝達関数

$$L(s) = K_p \cdot G_p(s) = \frac{K_p K}{s(\tau_m s + 1)}$$

K_p を上げるとゲイン交差周波数 ω_{gc} が上がり、位相余裕 (PM) は下がる。P 制御だけでは応答速度と安定性がトレードオフの関係にある

無駄時間の影響 / Dead Time Effect



制御ループの無駄時間

$\tau_d \approx 5$ ms (センサ処理 + 制御演算 + PWM 更新)

$$L_{delay}(s) = L(s) \cdot e^{-\tau_d s}$$

実用上 $PM \geq 60^\circ$ が必要

モデル上は安定なゲインでも、無駄時間を足すと実機で振動することがある

実習: L5 のレート P 制御 / Rate P-Control Recap

user_code.cpp (Lesson 5)

```
1 float Kp_rp=0.5f, Kp_yaw=2.0f, rmax=1.0f, ymax=5.0f;
2 float re = ws::rc_roll()*rmax - ws::gyro_x();
3 float pe = ws::rc_pitch()*rmax - ws::gyro_y();
4 float ye = ws::rc_yaw()*ymax - ws::gyro_z();
5 ws::motor_mixer(ws::rc_throttle(),
6     Kp_rp*re, Kp_rp*pe, Kp_yaw*ye);
```

L5 では飛ばして安定するかどうかだけを見た。 $K_p = 0.5$ は勘で決めた値である

モデルで見る L5 の設計 / Explaining L5's Gain

	Roll ($K = 102$)	Pitch ($K = 70$)
L5 の $K_p = 0.5$ での ζ	0.50	0.60
$\zeta = 0.7$ 設計の K_p	0.25	0.36

読み取り

L5 の $K_p = 0.5$ は Roll で $\zeta = 0.50$, やや振動的な設定だったとモデルが説明する。P 制御だけでは、ゲインを下げて安定性を確保すると定常偏差と外乱への弱さが残る。ここから I 項・D 項を追加する

PID 工学形式パラメータ / Engineering Form

パラメータ	意味	効果
K_p	比例ゲイン	大きいほど応答が速い, 大きすぎると振動
T_i	積分時間 [s]	小さいほど積分が強く効き偏差が早く消える
T_d	微分時間 [s]	大きいほど微分が強く効き振動を抑える
η	フィルタ係数	0.1 ~ 0.2, 大きいほどノイズに強い

教科書形式との対応

$$K_i = K_p / T_i, \quad K_d = K_p \cdot T_d$$

L8 の解答が使うゲイン (K_p, T_i, T_d) ($\eta = 0.125$ 共通) : Roll (0.25, 1.67, 0.01), Pitch (0.36, 1.67, 0.01)。 K_p は前ページの $\zeta = 0.7$ 設計値

Yaw は L8 の解答では経験値 $K_p = 2.0$ を使う。同じ設計式を最新の実測値 $K = 8.0$ に適用すると $K_p = 3.19$ になり, 設計式で追い直せる例として挙げる

D 項とアンチワインドアップ / D-Term & Anti-Windup

L8 学習者コード

理想 PID から不完全微分フィルタへ置換

```
1 float D = Kp*Td*(err-prev_err)/dt;  
2 // -> incomplete derivative filter  
3 float alpha = 2*eta*Td/dt;  
4 float a=(alpha-1)/(alpha+1);  
5 float b=2*Td/((alpha+1)*dt);  
6 d_filt = a*d_filt + b*(err-prev_err);
```

L8 では振動と定常偏差の体感を優先し、この 2 点は扱わない。sf_controller_pid/include/pid.hpp が vehicle 実装のコード

vehicle 実装: sf_controller_pid

微分は誤差ではなく**測定値**に掛ける (D-on-M)。目標値が階段状に変わっても微分キックが出ない

アンチワインドアップ

単純な積分クランプではなく、出力が飽和方向に押される間だけ積分を止める「条件付き積分」を使う

離散化と ARM 時リセット / Discretization & Reset

離散化: 双一次変換 (Tustin)

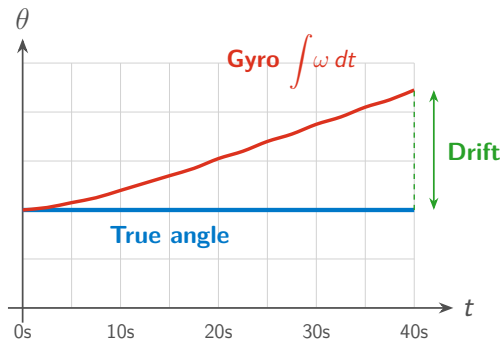
比例・積分・微分すべてに双一次変換を使う。L8 の不完全微分フィルタの式 (α, a, b) はこの変換そのもの

積分リセットのタイミング

積分器は DISARM 中ずっとゼロなのではなく、**ARM 遷移のたびに**リセットされる。着地して DISARM し、次に ARM した瞬間に積分が空になり、前のフライトの偏差を引きずらない

ライブチューニング (param set での即時反映) では、積分器の状態は**保持**される。ゲイン変更のたびにループを蹴らないための区別

姿勢推定: ジャイロドリフト問題 / Gyro Drift Problem

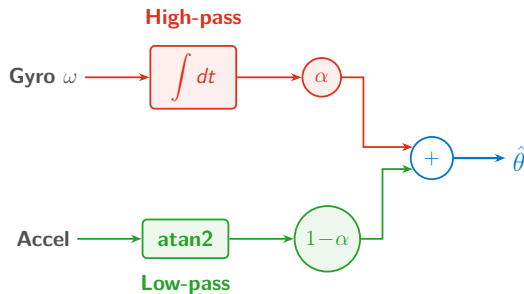


- ジャイロは角速度 ω を測定, 角度は積分 $\theta = \int \omega dt$ が必要
- バイアスが蓄積し, ドリフトが増大する

問題

ジャイロ単体では長時間の姿勢推定ができない。STABILIZE 以上で姿勢角を目標にするには, 角度そのものの推定が要る

相補フィルタと STABILIZE / Complementary Filter



数式

$$\hat{\theta}_k = \alpha(\hat{\theta}_{k-1} + \omega \Delta t) + (1 - \alpha)\theta_{accel}$$

- $\alpha = 0.98$ (ジャイロ 98% + 加速度 2%)
- StampFly の既定は 15 状態 ESKF。相補フィルタは自作して `estimated_roll()` と比較する教材

STABILIZE ではこの推定角を外側ループの入力にし，スティックが傾き角そのものを指令する

システム同定: sf sysid fit / Model Fitting

K_p が既知なら, 閉ループデータから開ループプラントを復元できる

$u(t) = K_p(\text{target} - \text{gyro})$, $y(t) = \text{gyro}$ として $G_p(s) = K/(s(\tau_m s + 1))$ に最小二乗フィット

出力例

```
1 sf sysid fit flight.csv --kp 0.5 --plot
2   Roll  K= 98.5 (ref:102.0, err:3.4%) tau_m=0.019 R2=0.94
3   Pitch K= 72.1 (ref: 70.0, err:3.0%) tau_m=0.021 R2=0.91
```

理論値 (L6) との誤差 3~4%は, モデル簡略化と無駄時間の影響で説明できる

自動チューニング: sf sysid rate-fit / rate-tune

ETFE によるプラント同定と仕様ベースのゲイン設計

チャープ（またはダブレット）励振で軸ごとに (b, L, T) を推定し, $G_p(s) = b e^{-Ls} / (s(Ts + 1))$ にフィットする。sf sysid fit と違い K_p を仮定しない開ループ同定

コマンド

```
1 sf sysid rate-fit flight.csv --axis roll -o fit.json
2 sf sysid rate-tune --fit fit.json --wc 25 --pm 60
```

rate-tune はゲイン交差周波数 ω_c と位相余裕 PM の仕様を満たす $K_p/T_i/T_d$ を逆算し, param set rate.roll.kp ... の形で出力する

注: 学習者コードで rate-fit/rate-tune を使うには, 角速度目標を計算した直後に

`ws::set_rate_target(roll,pitch,yaw)` を呼ぶ (L7 で追加, Data Stream の `rate_ref_*` に記録される)

デモ: 完成コードで飛行 / Demo: Final Code

デモの見方

見るだけ: ステップ応答の波形と設計値の一致を確認するだけで要点はつかめる。**一緒に打つ:** 手元の flight.csv で `sf sysid fit` を試す。**帰宅後に再現:** 本日は見るだけにして、後日 L5~L9 を通して動かす

手順

- ① PID 化した学習者コードで飛行し, `sf log wifi` でテレメトリ取得
- ② `sf log viz` でロール角速度のステップ応答を表示
- ③ モデル予測 ($\zeta = 0.7$ の応答波形) と重ねて比較

会場の通信環境に備え, 事前取得済みのログを保険として用意する

- **オートチューン**: `sf sysid rate-tune` と同じ ω_c ・PM 仕様のゲイン設計を機体上で行う `sf_autotune`。励振はステップドサインで、周波数点ごとの I/Q 積算から $G(j\omega)$ を測り、プラントフィットまでオンボードで完結させる
- **ESKF**: 相補フィルタで自作した推定と、機体が実際に使う 15 状態 ESKF の違い
- **高度・位置ループ**: 今日のレート/姿勢カスケードのさらに外側に高度・位置ループが乗る。S5 のシミュレータ活用で扱う

理論とコードの対応表 / Theory-to-Code Map

教科書表記	L8 学習者コード	vehicle 実装	Data Stream
比例ゲイン K_p	float Kp (無次元)	rate.roll.kp (Nm/(rad/s))	—
積分時間 T_i	float Ti	rate.roll.ti (s)	—
微分時間 T_d, η	float Td, eta	rate.roll.td, η 固定	—
姿勢ゲイン	— (L8 では未使用)	attitude.roll.kp/ti/td	—
目標角速度	target	rate_sp_roll	rate_ref_roll
角速度計測値	ws::gyro_x()	state.angular_rate[0]	gyro_x
アンチwindアップ	— (未実装)	条件付き積分 (pid.hpp)	—
ARM 時リセット	— (未実装)	PidController::reset()	—

レートループの K_p は、学習者コードでは duty 空間の無次元値、vehicle 実装では物理トルク単位（ミキサーが推力配分に変換する前の値）。数値を単純比較できない点が「学習者コードと vehicle 実装の違い」を最もよく表す

復習パス / Review Path

コマンド	レッスン	文書	Notebook
<code>sf lesson switch 5</code>	L5 レート P 制御	<code>rate_p_control.tex</code>	<code>05_rate_control_tuning.ipynb</code>
<code>sf lesson switch 6</code>	L6 システムモデリング	<code>system_modeling.tex</code>	<code>04_pid_theory.ipynb</code>
<code>sf sysid fit</code>	L7 システム同定	<code>system_identification.tex</code>	<code>07_system_identification.ipynb</code>
<code>sf lesson switch 8</code>	L8 PID 制御	<code>pid_control.tex</code>	<code>04_pid_theory.ipynb</code>
<code>sf lesson switch 9</code>	L9 姿勢推定	<code>attitude_estimation.tex</code>	<code>08_sensor_fusion.ipynb</code>

すべて `sf lesson build` → `sf lesson flash` で実機に反映する。--solution で模範解答に切り替えられる

チェックポイント / Checkpoint

確認事項

- ☐ $G_p(s) = K/(s(\tau_m s + 1))$ と実測パラメータの対応を説明できる
- ☐ P 制御の限界と, I 項・D 項が何を補うかを説明できる
- ☐ 不完全微分・アンチwindアップ・ARM 時リセットで学習者コードと vehicle 実装がどう違うか説明できる
- ☐ `sf sysid fit` の入出力と, 理論値との比較の読み方を理解した

次のセッション

S5: シミュレータ・解析ツールの活用と発展的テーマ (15:30 – 16:30)

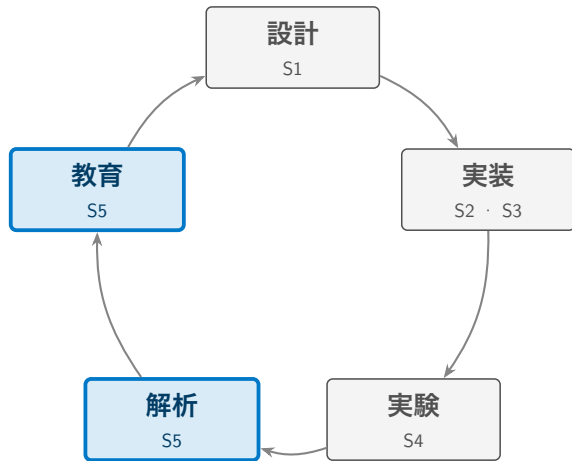
Session 5

シミュレータ・解析ツールの活用と発展的テーマ

15:30 – 16:30

SCI/SICE チュートリアル講座 2026

地図: 今ここ / Map: You Are Here



S4 で飛ばした結果を，本セッションではシミュレータと解析ツールで裏付ける。「実験」の次にくる「解析」と「教育」のフェーズにあたる。

本日の最後に扱うもの

3 層シミュレーション → SILS 実習 → 解析ツールと教育教材 → 研究・発展テーマ → 質疑

要点3つ / Three Key Points

- ① StampFly のシミュレーションは目的の異なる3層で構成され、層3 (SILS) は実ファームを無改変で走らせる
- ② 学習者が実装した推定器・制御器は、L1 Topic API 経由で既存の実装と差し替えられる
- ③ エコシステムは発展の途上にあり、研究・教育の両方で協力者を必要としている

本セッションの立ち位置

S4 までは1機の StampFly を飛ばす話だった。ここからは、飛ばさずに検証する方法と、1機から先の広がり扱う

シミュレーションの3層構造 / Three Simulation Layers



目的の異なる 3 層を使い分ける。詳細は `docs/architecture/simulation-policy.md` を参照

層1・層2: 設計モデルとログ再生 / Layers 1 & 2

層1: 設計用線形モデル

$G(s) = b e^{-Ls} / (s(Ts + 1))$ を `sf sysid rate-fit` で実飛行ログから同定し, 制御設計とループ整形の基準とする (S4 で扱った同定と同じ枠組み)

層2: 実ログ駆動オフライン再生

層1のモデルと Python に移植した制御則を, 実機ログから再構成した実外乱・実指令で駆動する閉ループ再生。パラメータ変更の A/B 判定の基準

`analysis/scripts/` 配下の Python 群。ヨー κ 修正のリプレー一致 0.0~0.7%, ALT_HOLD 再生誤差約 8%などの実績がある

Code Identity と Param Identity

MuJoCo（非線形 6 自由度の物理エンジン）と**実ファーム C++ そのもの**を，実機と同じソース・同じパラメータで走らせる

- 状態機械・推定器・フェイルセーフを含むシステム全体の検証ベンチ
- 推定器を ESKF から相補フィルタに差し替えても，ベンチは無改変で同じ検証ができる（アルゴリズムの中身に依存しない設計）
- シナリオファイルは 42 本，うち 33 本が `.expect` を伴う。32 本が CI 回帰（`sils-regression.yml`）の既定対象で，workshop 用の 1 本は手動実行

Code Identity を使った検証

実機ログに使うのと同じ同定パイプラインを, SILS が生成したログにもそのまま適用し, (b, L, T) を実機同定値と比較する (sf sils sysid-gate)

指標	許容差	2026-07-26 判定
ステップ応答立ち上がり時定数	$\pm 20\%$	Roll/Pitch PASS
gyro RMS	$\pm 50\%$	Yaw は識別が退化し FAIL

Yaw は反トルク零点を持ち, 現行の 3 パラメータフィットでは表現しきれない。全軸合格までは公称モデル 1 点最適化を避け, 摂動族での検証も併用する

VPython と Genesis / VPython & Genesis

	特徴	用途
VPython	軽量, ブラウザ 3D 表示, センサモデル充実	制御学習, SILS/HILS
Genesis	高精度物理エンジン, 2000 Hz 物理演算	高精度シミュレーション

コマンド

```
1 sf sim run vpython
2 sf sim run genesis
```

AtomS3 + Atom JoyStick を USB HID モードで使い, 実機と同じ操縦感でシミュレータを飛ばせる

sf sils コマンド一覧 / sf sils Subcommands

サブコマンド	機能
build	SILS ホストベンチをビルド
run	閉ループを実行しバンドルを書き出す
scenario	.scn シナリオを実行し合否判定
regression	全 .scn/.expect を実行し CI ゲート判定
gate	マイルストーンバンドルの合否確認
sysid-gate	モデル一致ゲート（前ページ）
gui	ブラウザ GUI（次ページ）

ESP-IDF は関与しない。firmware ソースをシステム GCC で素の CMake から直接コンパイルし、ヘッドレスで実行する

ブラウザだけで完結する SILS 実験環境

シナリオ作成・パラメータ編集・実行・グラフ化・飛行アニメーションをすべて画面で行える

起動

```
1 sf sils build
2 sf sils gui
```

- イベント（rc/wind/fault/handle など）を表で組んでシナリオを作る
- PID ゲインや ESKF 設定など 54 項目をブラウザで編集，変更した項目だけが反映される（再ビルド不要）
- three.js によるライブ 3D と Plotly のグラフが時刻カーソルで同期

学習者コードを SILS で試す / Learner Code in SILS

実機の前にシミュレータで確かめる

S4 で書いた PID コードを，実機を飛ばす前に SILS で検証する。墜落のリスクなしに極端なゲインや外乱を試せる

コマンド

```
1 sf sils build --target workshop
2 sf sils scenario ../workshop_acro.scn --target workshop
```

現状

--target workshop に対応済み。L5/L8 の模範解答で ARM→離陸→ホバー相当の飛行を確認できる。高度ループがないため着陸は DISARM による降下のみ。sf sils regression の自動 CI には未組み込み

デモ: シミュレータを動かす / Demo: The Simulator

デモの見方

見るだけ: 3D 上での挙動とグラフの対応を確認するだけで要点はつかめる。**一緒に打つ:** 手元でも `sf sim run vpython` を起動してみる。**帰宅後に再現:** 本日は見るだけにして、後日ジョイスティックで操縦する

手順

- ① `sf sim run vpython` でブラウザ 3D 操縦
- ② `sf sils gui` でシナリオを 1 本実行し、PASS/FAIL ゲートを確認
- ③ 同じシナリオでパラメータを変え、挙動の変化を見る

解析ツール: sf log analyze / viz

取得したテレメトリをそのまま解析する

sf log wifi/sf log capture で取得した CSV を、追加スクリプトなしで解析・可視化できる

コマンド

```
1 sf log viz flight.csv
2 sf log analyze flight.csv
```

詳しい手順は docs/guides/flight-log-viz.md。取得 → 可視化 → 解析の流れは S4 のシステム同定と同じログを使い回せる

15本の Jupyter Notebook

analysis/notebooks/education/ 配下に基礎から実装まで通しで並ぶ。
lib/stampfly_edu (シミュレーション・ダイナミクス・ESKF・ウィジェット) が
共通基盤

04_pid_theory.ipynb	S4 の PID 設計に対応
07_system_identification.ipynb	S4 のシステム同定に対応
08_sensor_fusion.ipynb	S4 の姿勢推定に対応
15_analysis_toolkit.ipynb	本セッションの解析ツールに対応

センサノイズの評価には `examples/education/noise_analysis/allan_variance.py`
で Allan 分散を使う。ホバリング精度のトリム調整には `sf trim` がある

自分の推定器・制御器を載せる

L1 Topic API (`sf::api::*`) は Pub-Sub トピックを `subscribe/publish` し, `IEstimator/IController` を実装して既存のコンポーネントと差し替えられる

- 推定: `sf_estimator` (インターフェース) + `sf_estimator_eskf`.
`sf_estimator_complementary` という代替実装もすでにある
- 制御: `sf_controller` (インターフェース) + `sf_controller_pid`
- Workshop 受講者の L0 と同じ機体・同じファーム基盤の上で, より深いレイヤーにアクセスできる

再現性の3原則 / Three Principles of Reproducibility

原則	内容
Code Identity	SILS は実ファームと同じソースをそのままコンパイルして走る
Parameter Identity	同じパラメータで走らせる
Model Fidelity	実機飛行後にモデルをログで較正する（後追いの精度向上）

パラメータは `param set/param save` で NVS に保存する。自作の推定器・制御器を試すときも、この3原則があるから「SILS で通った」が実機の挙動を意味する

- **AI コーディングエージェントとの協働:** このエコシステムのファームウェア・シミュレータ・スライド・教材文書は, AI コーディングエージェントとの協働で構築している。StampFly を題材にした AI コーディング教育の教材化も構想している
- **強化学習と Sim-to-Real:** シミュレータ上で学習した制御方策を実機に転移する流れを教材化できれば, 古典制御から学習型制御へカリキュラムの射程が広がる。機体が小型・軽量で墜落時のリスクが小さいことは Sim-to-Real の実機検証に向く

発展テーマ: 制御・推定・自律飛行 / Autonomy

- **MPC・適応制御**: 今日扱った PID の先にある発展的な制御手法。エコシステムの今後の拡張先として構想している
- **ESKF・オートチューン**: 15 状態 ESKF の実装, S4 で紹介した `sf_autotune` によるオンボードチューニング
- **高度・位置ループ**: レート/姿勢カスケードの外側に `ALT_HOLD/POS_HOLD` が乗る。外乱オブザーバ (`altitude.dob.fc`) もパラメータで opt-in できる
- **Tello 互換 SDK と ROS2**: Python SDK (`lib/stampfly`) と `sf flight` で自律飛行の入口を用意している。ROS2 連携は現時点では構想段階 (実装は未着手)

教育現場での実績

- 4+1 日間ワークショップ
- 大学講義（半期 15 回シラバス）
- 高校教員向け DXH 講座
- Blockly (sf blocks --demo) や競技会 (sf competition) も授業向けに用意している

コミュニティへ

本エコシステムはまだ発展の途上にあり、構想はあっても実装が追いついていない部分も多い。開発や教材改善に協力してくれる仲間を探している。OSS として公開している理由の一つはそこにある

理論とコードの対応表 / Concept-to-Code Map

概念	実装場所	コマンド
層 1: 設計用線形モデル	control/models/stampfly_physical.yaml	sf sysid rate-fit
層 2: 実ログ駆動オフライン再生	analysis/scripts/	—
層 3: SILS	simulator/sils/	sf sils scenario
モデル一致ゲート	tools/log_analyzer/rate_sysid.py	sf sils sysid-gate
推定インターフェース	sf_estimator / sf_estimator_ekf	—
制御インターフェース	sf_controller / sf_controller_pid	—

3層とも同じ物理パラメータ (stampfly-parameters.md) を参照する。層を追うごとにプラント物理・制御コードの忠実度が上がる

復習パス / Review Path

コマンド	扱う内容	文書	Notebook
<code>sf sim run vpython</code>	VPython シミュレータ	<code>simulator/README.md</code>	—
<code>sf sils gui</code>	SILS ブラウザ実験環境	<code>simulator/sils/gui/README.md</code>	—
<code>sf log viz</code>	ログ可視化	<code>docs/guides/flight-log-viz.md</code>	<code>15_analysis_toolkit.ipynb</code>
—	シミュレーション方針	<code>docs/architecture/simulation-policy.md</code>	—
—	教育カリキュラム	<code>analysis/notebooks/README.md</code>	01～15 (全 15 本)

想定質問

- モデル一致ゲートで Yaw だけ FAIL しているのはなぜか
- 自作の制御器を載せるとき、安全機構（フェイルセーフ）はどこまで自分で書く必要があるか
- ROS2 連携はいつ頃実装される見込みか

質疑の時間は本セッション末の 15 分。時間内に扱いきれなかった質問は、リポジトリの Issue で受け付けている

チェックポイント / Checkpoint

確認事項

- ☐ シミュレーションの3層構造と、それぞれの役割を説明できる
- ☐ `sf sils gui` でシナリオ実行とパラメータ編集を体験した
- ☐ L1 Topic API で自分の推定器・制御器を差し替える方法を理解した
- ☐ 教育 Notebook と解析ツールの入口を把握した

本日はこれで終了

資料・録画は公開されている。持ち帰り資料（付録）で復習を続けられる

Session 付録

チートシートと復習パス

持ち帰り資料

SCI/SICE チュートリアル講座 2026

sf CLI チートシート (1/2) / sf CLI Cheat Sheet

コマンド	機能
<code>sf doctor</code>	環境診断（問題があればまず実行）
<code>sf build/flash/monitor</code>	ビルド・書き込み・シリアルモニタ
<code>sf telemetry</code>	50 Hz テレメトリのライブ表示（ <code>--web</code> でブラウザ）
<code>sf blocks</code>	Blockly ブロックプログラミング（ <code>--demo</code> で機体なしデモ）
<code>sf cal gyro/accel/mag</code>	センサキャリブレーション
<code>sf lesson</code>	Workshop レッスン管理（ <code>switch/build/flash</code> ）
<code>sf trim</code>	ホバーログから姿勢トリムを算出
<code>sf flight</code>	Tello 風ミニコマンド （ <code>takeoff/land/hover/jump</code> 等）
<code>sf competition</code>	ホバー耐久・スコア記録

sf CLI チートシート (2/2) / sf CLI Cheat Sheet

コマンド	機能
<code>sf log wifi/capture</code>	WiFi/USB 経由でテレメトリ・ログ取得
<code>sf log viz/analyze</code>	ログ可視化・解析
<code>sf sim run vpython/genesis</code>	シミュレータ起動 (S5)
<code>sf sils build/scenario/gui</code>	SILS ベンチ (S5)
<code>sf sysid fit</code>	閉ループ P 制御ログからプラント同定 (L7)
<code>sf sysid rate-fit/rate-tune</code>	開ループ ETFE 同定と仕様ベース PID 設計 (S4)
<code>sf params check</code>	物理パラメータ整合検査

順序は `sf --help` 表示順に準拠。全コマンドは `lib/sfcli/commands/` に実装がある

カテゴリ	関数
モータ制御	<code>motor_set_duty(id,duty),</code> <code>motor_set_all(duty),</code> <code>motor_stop_all(),</code> <code>motor_mixer(t,r,p,y), arm(),</code> <code>disarm(), is_armed()</code>
コントローラ入力 ボタン・モード	<code>rc_throttle/roll/pitch/yaw()</code> <code>rc_throttle_yaw_button(),</code> <code>rc_roll_pitch_button(),</code> <code>rc_stabilize_acro_mode(),</code> <code>rc_alt_mode(), rc_pos_mode()</code>
LED 制御	<code>led_color(r,g,b),</code> <code>disable/enable_led_task()</code>

カテゴリ	関数
IMU センサ	gyro_x/y/z(), accel_x/y/z()
環境・距離センサ	baro_altitude/pressure(), mag_x/y/z(), tof_bottom/front(), flow_vx/vy/quality()
姿勢推定	estimated_roll/pitch/yaw(), estimated_altitude()
制御目標のロギング	set_rate_target(r,p,y), set_angle_target(r,p) (L7, Data Stream の rate_ref_*/angle_ref_* に記録, 制御に は無関係)
ユーティリティ	millis(), battery_voltage(), print(fmt,...), set_channel(ch)

#include "workshop_api.hpp", 全関数は ws:: 名前空間

トラブルシューティング (1/2) / Troubleshooting

症状	原因	対処
StampFly の WiFi が 見つからない	電源が入っていない	バッテリーを確認、電源を入れ直す
接続しても通信できない	IP アドレスが違う	192.168.4.1 を確認
離陸しない	キャリブレーション未実施	<code>sf cal gyro</code> を実行
振動する シミュレータに自動 フォールバック	PID ゲインが高すぎる WiFi 未接続	ゲインを下げる (S4 参照) 意図的ならそのまま使用可

出典: docs/guides/troubleshooting.md

トラブルシューティング (2/2) / Troubleshooting

症状	原因	対処
ビルドエラー（赤字 <code>error:</code> <code>sf lesson</code> <code>flash</code> が失敗 モータが回らない	打ち間違い（カンマ・ セミコロン抜け等） USB ケーブル・ポート の問題 ARM し忘れ / USB 接続のまま	エラー 1 行目のファイル名・行番号を確認し 実習コードと見比べる ケーブル交換 / ポートを使う他アプリを閉じ る / BOOT ボタンを押しながら挿し直す 機体ボタンを 1 回クリック / USB を外し バッテリー駆動を確認
起動 LED が緑になら ない	校正中に機体を動か した	機体を静かに置き直し、緑点灯 + ビープ 3 回 まで待つ

出典: DXH 講座トラブルシューティング早見表 (docs/workshop/dxh2026/handout.md)

5 セッション復習パス / Review Path, All Sessions

	コマンド	扱う内容	文書	Notebook
S1	<code>sf sim run</code>	シミュレータ操縦	<code>docs/architecture/simulation-policy.md</code>	01_hello_stampfly
S1	—	大学講義シラバス	<code>docs/university/syllabus.md</code>	全 15 本
S2	<code>sf lesson switch 4</code>	IMU センサ	<code>firmware/vehicle/docs/architecture.md</code>	01_hello_stampfly
S2	<code>sf log wifi/viz</code>	テレメトリ取得と可視化	<code>docs/guides/flight-log-viz.md</code>	15_analysis_toolkit
S3	<code>sf lesson switch 1</code>	モータ制御	<code>firmware/vehicle/docs/hardware_init.md</code>	—
S3	<code>sf lesson switch 2</code>	コントローラ入力	<code>protocol/spec/ (ControlPacket)</code>	—
S4	<code>sf lesson switch 8</code>	PID 制御	<code>pid_control.tex</code>	04_pid_theory
S4	<code>sf sysid fit</code>	システム同定	<code>system_identification.tex</code>	07_system_identification
S5	<code>sf sils gui</code>	SILS ブラウザ実験環境	<code>simulator/sils/gui/README.md</code>	—
S5	<code>sf log viz</code>	ログ解析	<code>docs/guides/flight-log-viz.md</code>	15_analysis_toolkit

持ち帰りのご案内 / Take It Home

資料はすべて公開されている

- リポジトリ: https://github.com/M5Fly-kanazawa/stampfly_ecosystem
- ドキュメント:
https://m5fly-kanazawa.github.io/stampfly_ecosystem/docs/
- 本日の内容はオンデマンド配信でも後日視聴できる

ご自身の StampFly で

ノート PC と StampFly さえあれば、今日のデモは自宅・研究室でそのまま再現できる。まずは `sf doctor` で環境を確認してから始めてほしい

ご質問・不具合報告はリポジトリの Issue へ。開発に協力してくれる方も歓迎する